

Ma petite méthode de décomposition en 4 ème Forme Normale

(Mes petites pompes pour l'examen de modélisation base de données)

1- le squelette d'entités

- **Le plus important est le squelette d'entités**, tant qu'il me reste des passages pas trop clairs je continue à l'améliorer.

Pour trouver le squelette :

- je cherche les entités
- je les assemble dans un squelette d'entité.
- pour vérifier mon squelette 3 étapes que je répète jusqu'à trouver un squelette qui me convienne :
 - vérifier que mes entités en sont bien (je dois pouvoir citer 2 attributs si c'est une entité)
 - vérifier les sous entités dans le squelette (pour un A je n'ai qu'un seul B)
 - vérifier que tout ce que je dois modéliser est bien dans mon squelette (les entités, les attributs mais également les relations entre tout cela)
- les **seuls attributs** dans le squelette sont les attributs isolés (qui ne dépendent d'aucune entité)

Remarques :

- les barres verticales symbolisent les attributs des entités
- une entité A a une sous-entité B si pour un A il n'y a qu'un seul B
- dans le squelette on ne s'intéresse qu'à l'unicité,
- quand ça marche pas d'en un sens j'essaye dans l'autre

Quand et seulement quand mon squelette est parfait je passe à la suite

2 - les patates

J'entoure sur le squelette d'entités les entités qui sont en relation. Cela me donne un certain nombre de patates qu'il faut maintenant que je relis avec des dépendances.

3 – Les Dépendance Fonctionnelles

Je cherche les dépendances fonctionnelles d'entités, il y a **une DF par entité**.

Je cherche ensuite les dépendances fonctionnelles qui permettent de relier entre elles les patates. Elles seront obligatoirement de la forme $A, B \rightarrow C$

4 - Les Dépendances Multivaluées

S'il me reste des patates qui ne sont pas reliées entre elles, je cherche des dépendances multivaluées qui le feraient.

Pour vérifier si une multivaluée est vraie (c'est à dire indépendante des autres attributs) je peux utiliser la méthode des triplets ou utiliser la phrase magique. Pour vérifier que $X \twoheadrightarrow Y$ j'essaye de faire la phrase suivante :

Si pour un X je peux avoir plusieurs Y est-ce indépendant de Z qui...

en essayant de finir la phrase au plus vite sur le Y. Si j'arrive à rejoindre le Y sans passer par X alors elle est dépendante (donc faux) sinon (si je dois passer par X pour rejoindre le Y) c'est indépendant de Z, il me faut encore vérifier avec tous les Z (les autres attributs).

Pour savoir avec quels « autres Z » à tester, je me base sur le squelette d'entités. Je ne teste que par rapport aux patates. Si X et Y sont des patates, je dois tester avec toutes les autres patates (Z).

Une fois que toutes mes patates sont reliées (avec des DF ou des DM) je passe à l'arbre. Je ne cherche pas plus de DM que nécessaire !

5 – l'arbre de décomposition

5.1 – la relation universelle (de départ)

$$R(A(...), C(...), D) \{d1, d2, d3, dm1, dm2\}$$

Pour l'écrire je me base sur le squelette d'entités. Il contient tout, c'est donc l'ensemble des patates que je dois retrouver comme attributs de cette relation.

Si A est une entité et B une de ses sous-entité, on peut écrire l'ensemble des attributs ainsi :

id-A(...) ce qui est équivalent à id-A(...id-B(...))

Id-A et id-B sont les clés des entités A et B.

id-A(...) se lit : l'attribut id-A et tous les attributs qu'il détermine (y compris par transitivité)

id-A(...id-B(...)) se lit : l'attribut id-A et tous les attributs qu'il détermine dont id-B et tout les attributs qu'il détermine.

Cela nous permettra d'écrire dans la décomposition : id-A(...id-B(-))

qui se lit : l'attribut id-A et tous les attributs qu'il détermine dont id-B **sauf** les attributs déterminés par id-B.

La relation de départ contient toutes les dépendances (DF et DM)

5.2 – l'algorithme de décomposition

Tant qu'une relation n'est pas en 4NF, je la décompose en 2 relations en utilisant une dépendance.

S'il me reste une vraie DM je commence par elle.

Cela me crée une **première relation** qui contient **tous les attributs de la dépendance**

et une **seconde relation** qui contient **tous les attributs de la relation** que je décompose **moins la partie droite de la dépendance**

je répartir ensuite les dépendances de la relation que je décompose dans les 2 relations que je viens de créer.

Attention : Quand j'utilise une DM pour décomposer elle disparaît (en fait elle s'est transformée en relation). Quand c'est une DF que j'utilise, on la retrouve au moins dans la première relation générée.

Attention : il n'existe pas de relation avec un seul attribut

Attention : il ne peut pas y avoir de branche vide dans l'arbre

Comment vérifier qu'une relation est en 4NF :

- Si elle a aucune dépendance , **elle est en 4NF**

- Si elle a au moins 2 dépendances (ayant des choses différentes à gauche) **elle n'est pas en 4NF**

- Quand elle a une seule dépendance, il faut que je cherche la clés de la relation et je vérifie si sa dépendance est de la forme « la clés détermine des attributs ». Si c'est le cas elle est en 4NF sinon non.

5.3 Fin de l'arbre

Toutes les relations en bout de branches sont donc en 4NF (sinon il faut continuer à décomposer).

Pour chacune :

- je souligne les attributs clés

- je l'encadre

Il ne reste plus qu'une seule étape, éliminer les relations redondantes. Si une des relations de ma solution est incluse dans (ou égale à) une autre, je la supprime.

Les relations qui restent sont alors votre solution. Une table sera créée pour chacune d'elles et cela sera votre **base de données, non redondante et sans perte d'informations**